Universidad de San Andrés

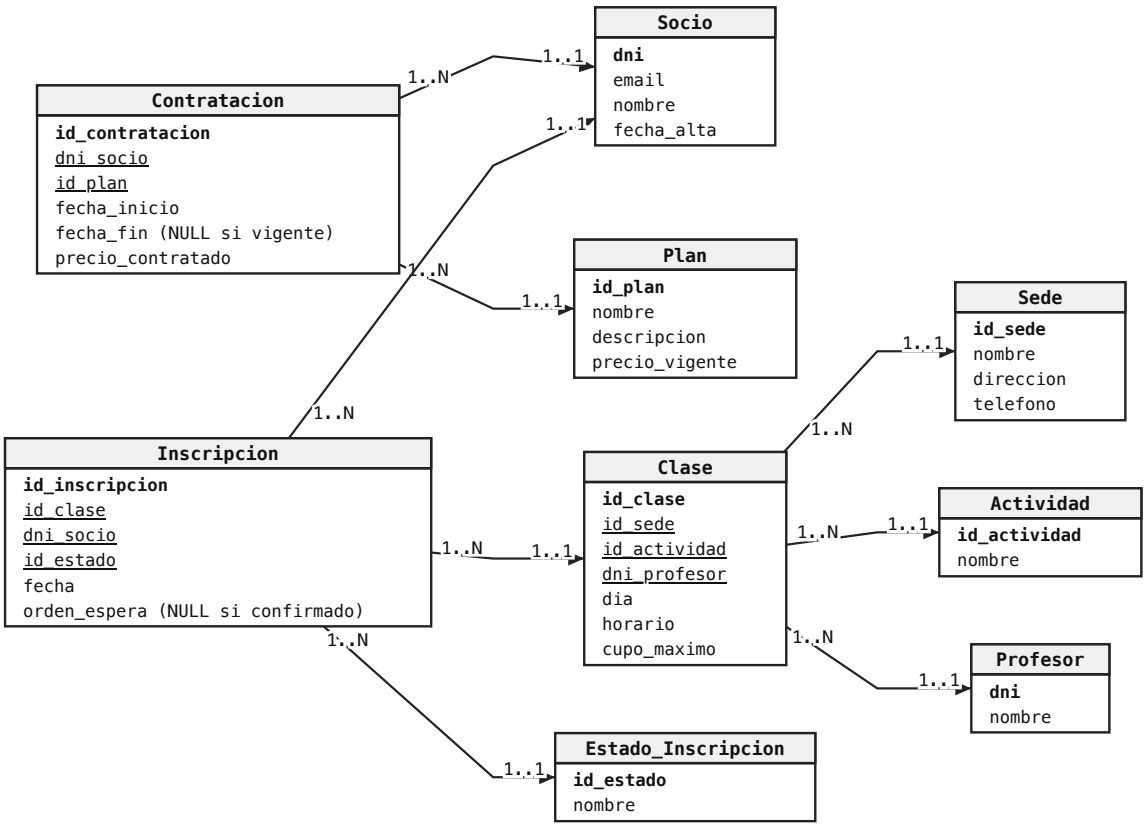
Solución — Guía Extra de Repaso — Parcial

Gestión y Arquitectura de Datos, Universidad de San Andrés

Convenciones: como en la guía de DER, la clave primaria va en **negrita** y las claves foráneas subrayadas. Cada flecha sale de la tabla que tiene la FK hacia la tabla referenciada, con las cardinalidades en los extremos (`1..N` del lado de la FK, `1..1` del lado referenciado). En DER hay muchas soluciones válidas: lo importante es que las claves y las cardinalidades estén bien y que el modelo soporte todos los requisitos del enunciado. Todas las queries de SQL fueron ejecutadas contra la base del enunciado en SQLite; los resultados que se muestran son salidas reales.

Parte 1 — Diagramas Entidad-Relación

Ejercicio 1 — Cadena de Gimnasios

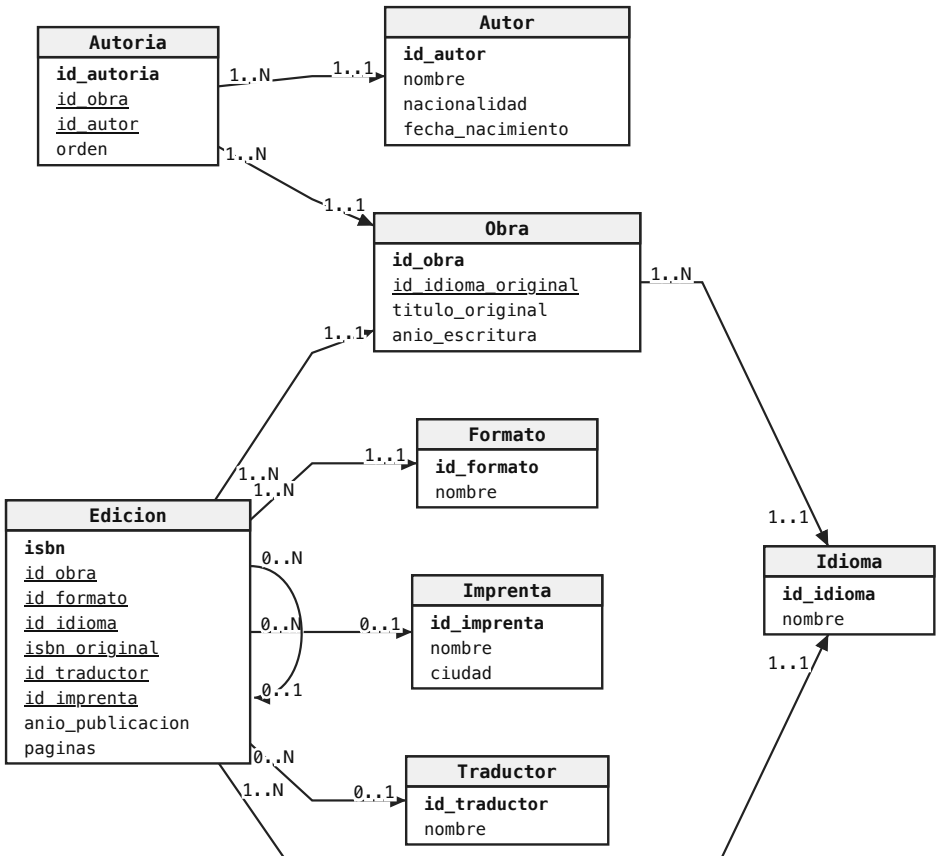


Notas:

- **Pregunta:** una FK `id_plan` en `Socio` solo guarda el plan actual: al cambiar de plan se pisa el anterior y se pierde el historial. La entidad intermedia `Contratacion` con fechas lo resuelve; el plan vigente es la contratación con `fecha_fin` en NULL (a lo sumo una por socio, restricción que se controla en el backend).
- **Pregunta (segunda parte):** el precio de `Plan` es el vigente y cambia con el tiempo. Por eso `precio_contratado` se **copia** en la contratación (snapshot): si solo estuviera en `Plan`, al actualizarlo perderíamos cuánto se le cobró históricamente a cada socio.
- No hace falta una relación directa profesor–sede: queda determinada a través de las clases que dicta.

- `orden_espera` es NULL para los inscriptos confirmados. Alternativa igual de válida: separar `Inscripcion y Lista_Espera` en dos entidades.

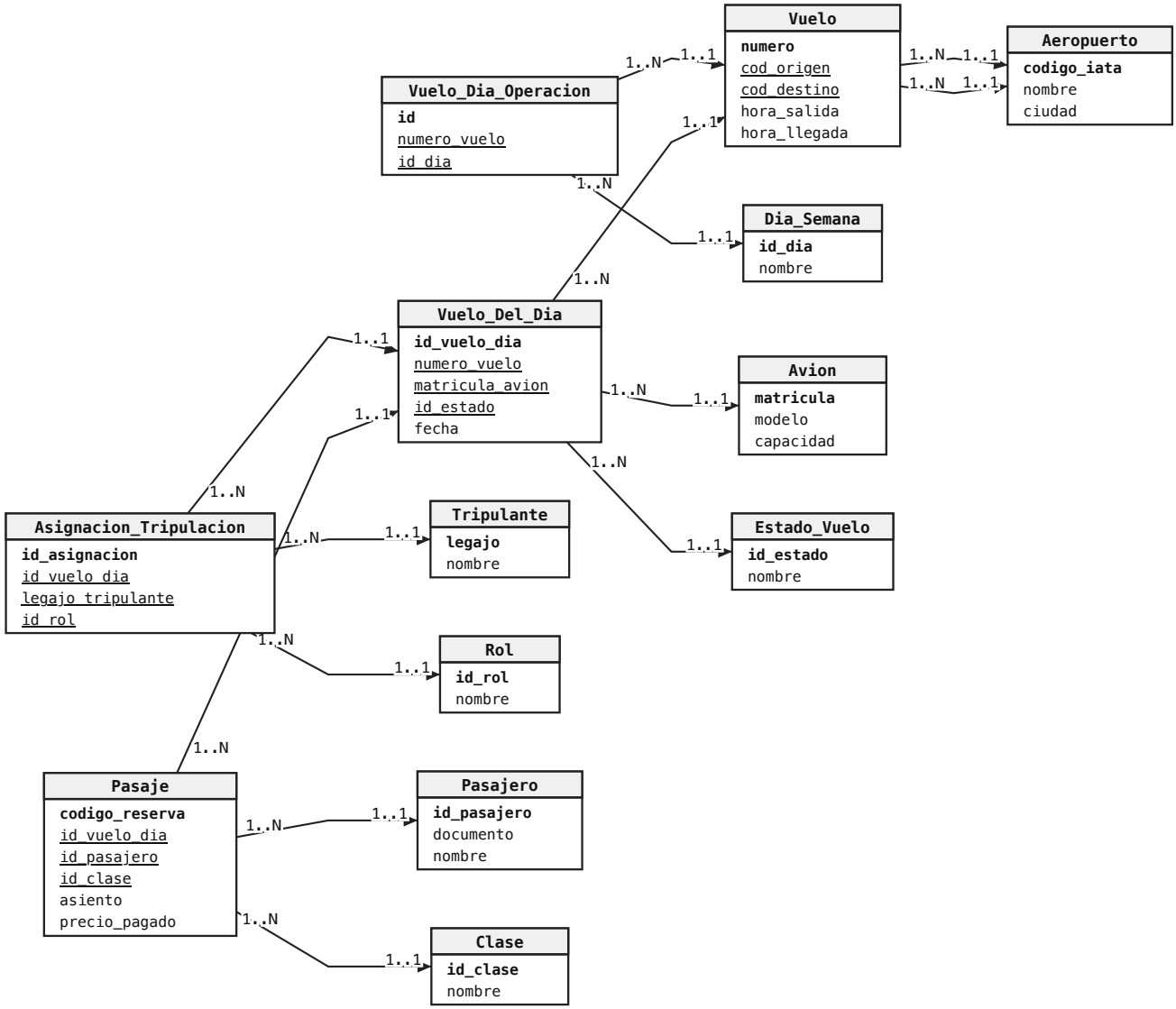
Ejercicio 2 — Editorial



Notas:

- **Pregunta:** el ISBN identifica a la **edición**, no a la obra. Ejemplo: «Rayuela» en tapa dura y en bolsillo son la misma obra con dos ISBN distintos. Si el ISBN fuera atributo de la obra, no podríamos registrar más de una edición (o tendríamos que repetir toda la obra por cada edición, duplicando datos).
- La traducción es una **autorreferencia**: `isbn_original` apunta a otra fila de `Edicion` (la flecha que sale y vuelve a la misma entidad). Tanto `isbn_original` como `id_traductor` son NULL para ediciones que no son traducciones.
- `id_imprenta` es NULL para los ebooks. Alternativa válida (estilo tipo/subtipo, como los repartidores del parcial viejo): subtipos `Edicion_Fisica` (con imprenta) y `Edicion_Digital`, mutuamente excluyentes.
- El orden de autoría va en la entidad intermedia `Autoria`: es un dato del par obra-autor, no de ninguno de los dos por separado.

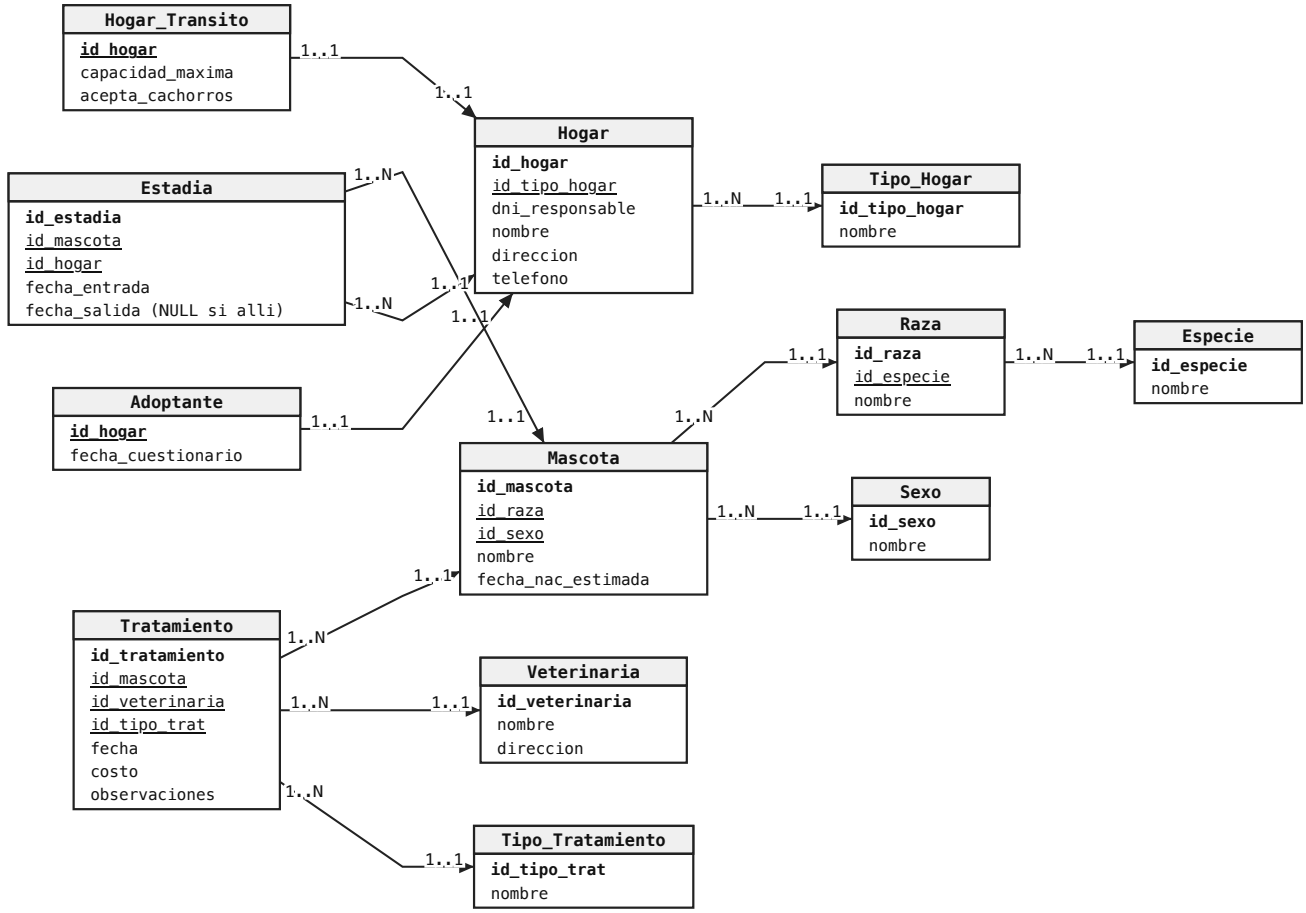
Ejercicio 3 — Aerolínea



Notas:

- **Pregunta (plantilla vs. instancia):** AR1304 vuela todos los lunes, miércoles y viernes: es **un** vuelo pero muchas operaciones. Si fecha, avión y estado vivieran en `Vuelo`, cada día pisaría los datos del anterior y no habría historial; y un pasaje «para AR1304» no diría a qué fecha corresponde. Por eso se separa la plantilla (`Vuelo`) de la instancia (`Vuelo_Del_Dia`). Es el mismo patrón que Materia/Dictado en la guía de DER.
- `Vuelo` tiene **dos FKs a Aeropuerto** (origen y destino): dos flechas a la misma entidad.
- El **rol va en la asignación**, no en el tripulante: depende del par tripulante–vuelo del día (hoy copiloto, mañana comandante).
- Los días de operación son un atributo multivaluado → tabla propia `Vuelo_Dia_Operacion`.
- «Un asiento no se vende dos veces»: restricción de unicidad compuesta `UNIQUE(id_vuelo_dia, asiento)` sobre `Pasaje` (el modelo solo no la garantiza).
- `precio_pagado` es un snapshot: dos pasajeros del mismo vuelo pueden haber pagado distinto.

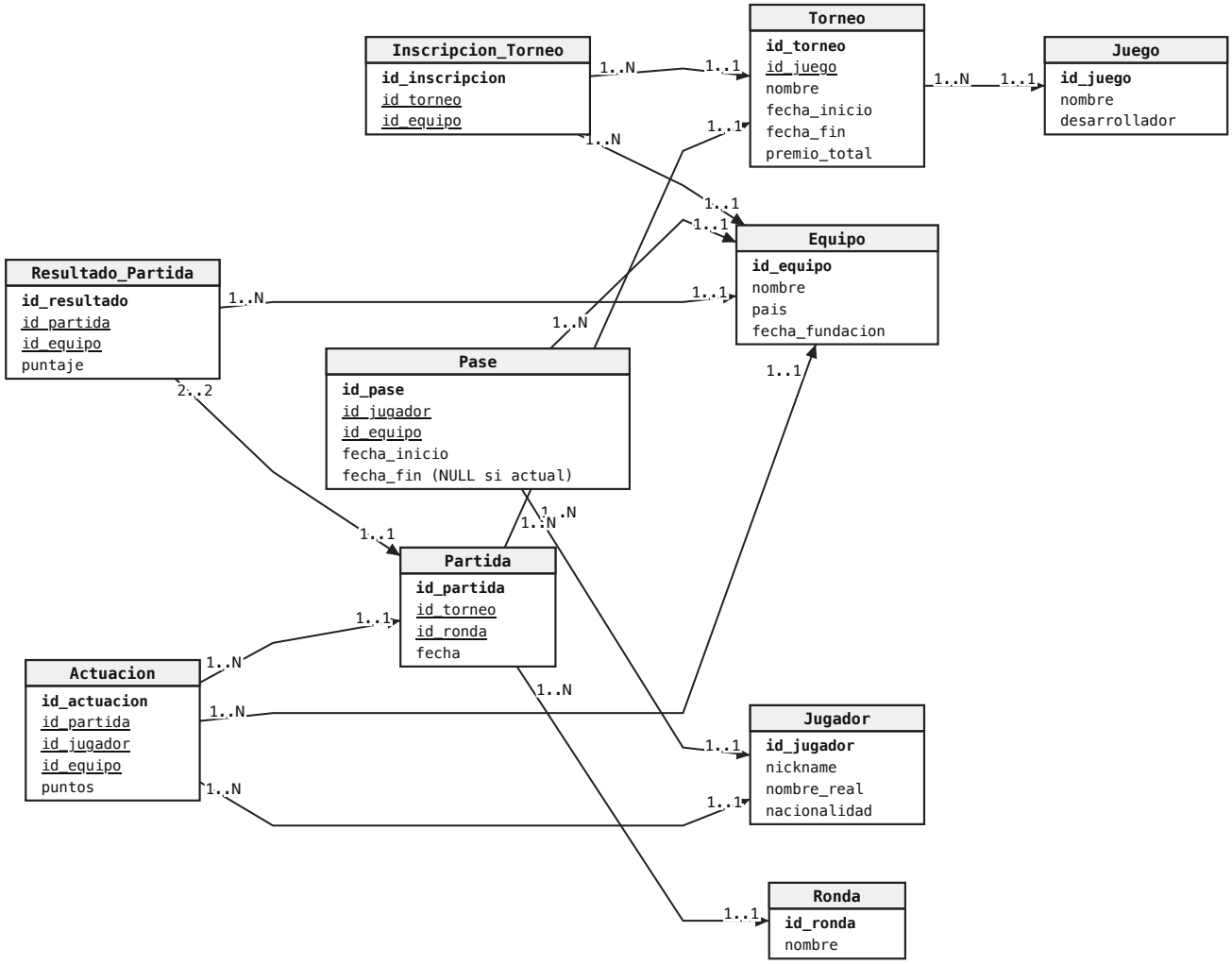
Ejercicio 4 — Refugio de Mascotas



Notas:

- Estructura **tipo/subtipo** igual que los repartidores del parcial viejo: los atributos comunes van en `Hogar` y los específicos en `Hogar_Transito` / `Adoptante`, cuya PK es a la vez FK al padre (por eso la cardinalidad 1..1 → 1..1). La exclusividad mutua (un hogar no puede ser de los dos tipos) se apoya en `id_tipo_hogar` y se controla en el backend.
- El historial de hogares es la entidad `Estadia` con `fecha_entrada` / `fecha_salida`; la estadía vigente es la que tiene `fecha_salida` NULL.
- **Pregunta:** el modelo **no** garantiza que una mascota tenga una sola estadía abierta: nada impide insertar dos filas con `fecha_salida` NULL para la misma mascota. Es una restricción de negocio que se controla en el backend (o con un índice parcial/trigger en el motor).
- Cada aplicación de tratamiento es una fila de `Tratamiento`: relaciona mascota, veterinaria y tipo, con sus datos propios (fecha, costo, observaciones).

Ejercicio 5 — Torneos de E-sports



Notas:

- «Exactamente dos equipos»: con `Resultado_Partida` (una fila por equipo por partida, cardinalidad 2..2) el modelo en sí permite 1 o 3 equipos; la cantidad exacta se controla en el backend. Alternativa válida: poner `id_equipo_A`, `id_equipo_B`, `puntaje_A` y `puntaje_B` directamente en `Partida` (dos FKs a la misma entidad, como origen/destino en la aerolínea); garantiza los dos equipos pero es más rígida para consultar.
- El historial de transferencias es `Pase`, con el equipo vigente identificado por `fecha_fin` NULL — mismo patrón que las contrataciones del gimnasio.
- **Pregunta:** en principio, el equipo con el que jugó alguien se deduce cruzando la fecha de la partida con el historial de pases. Pero es frágil: si un pase se carga con fechas incorrectas, o hay una transferencia el mismo día de la partida, la deducción falla. Por eso `Actuacion` lleva `id_equipo` explícito: es un **snapshot** del momento de la partida, igual que el precio en un detalle de pedido.

Parte 2 — Normalización

Ejercicio 6 — Turnos Médicos

a) Dependencias funcionales

- DNI_Paciente → Nombre_Paciente, Email_Paciente
- (DNI_Paciente, Fecha_Hora) → Matricula_Medico
- Matricula_Medico → Nombre_Medico, Especialidad
- Especialidad → Precio_Consulta, Piso

b) Clave primaria mínima

(DNI_Paciente, Fecha_Hora): un paciente tiene a lo sumo un turno por fecha y hora, y ningún atributo por separado alcanza para identificar la fila.

c) Pasaje a 3FN

La forma directa de llegar a 3FN: mirar las dependencias de a) y darle una tabla propia a cada lado izquierdo que no sea la clave completa.

- Nombre y email dependen solo del DNI → tabla **Paciente**.
- Nombre del médico y especialidad dependen de la matrícula → tabla **Medico**.
- Precio y piso dependen de la especialidad → tabla **Especialidad**.
- Lo que queda es el turno en sí: quién, cuándo y con qué médico.

Paciente

DNI (PK)	**Nombre**	**Email**

Especialidad

Id_Especialidad (PK)	**Nombre**	**Precio_Consulta**	**Piso**

Medico

Matricula (PK)	**Nombre**	**Id_Especialidad (FK)**

Turno

DNI_Paciente (PK, FK)	**Fecha_Hora (PK)**	**Matricula_Medico (FK)**

> El error típico acá es separar Médico y frenar ahí: si el precio y el piso quedan en `Medico`, siguen repetidos para todos los médicos de la misma especialidad. La cadena es doble: Turno → Médico → Especialidad → (precio, piso), y hay que cortarla entera.

Ejercicio 7 — Facturación

a) Dependencias funcionales

- (Nro_Factura, Renglon) → Cod_Producto, Cantidad, Precio_Unitario
- Nro_Factura → Fecha, CUIT_Cliente
- CUIT_Cliente → Razon_Social
- Cod_Producto → Descripcion_Producto
- Subtotal y Total_Factura son derivados (se calculan a partir de los demás).

b) Clave primaria mínima

(Nro_Factura, Renglon).

c) Pasaje a 3FN

- Fecha y cliente dependen solo del número de factura → tabla **Factura**.
- La razón social depende del CUIT → tabla **Cliente**.
- La descripción depende del código de producto → tabla **Producto**.
- `Subtotal` y `Total_Factura` se calculan → se eliminan.
- Queda el renglón: qué producto, cuánto y a qué precio se vendió.

Cliente

CUIT (PK)	**Razon_Social**

Factura

Nro_Factura (PK)	**Fecha**	**CUIT_Cliente (FK)**

Producto

Cod_Producto (PK)	**Descripcion**	**Precio_Lista**

Detalle_Factura

Nro_Factura (PK, FK)	**Renglon (PK)**	**Cod_Producto (FK)**	**Cantidad**	**Precio_Unitario**

d) ¿Dónde queda Precio_Unitario?

En **Detalle_Factura**. No depende solo de `Cod_Producto`: es el precio *al momento de la venta* (snapshot). Si lo lleváramos a `Producto`, al actualizar el precio de lista cambiarían retroactivamente todas las facturas históricas — un error grave del negocio, no solo de modelado. El precio de lista actual es **otro dato** y va en `Producto` como `Precio_Lista`.

e) ¿Qué se hace con Subtotal y Total_Factura?

Son **derivados**: se eliminan y se calculan al consultar (`SUM(Cantidad * Precio_Unitario)`). Guardarlos sería redundancia con riesgo de inconsistencia; si por rendimiento se decidiera persistirlos, sería una desnormalización deliberada y habría que mantenerlos sincronizados.

Ejercicio 8 — Empleados

a) Dependencias funcionales

- Legajo → DNI, CUIL, Nombre, Email, Id_Oficina
- DNI → Legajo (y por lo tanto todo el resto); ídem CUIL e Email
- Id_Oficina → Piso_Oficina

b) Claves candidatas y clave primaria mínima

Hay **cuatro claves candidatas**, todas de un solo atributo: Legajo, DNI, CUIL y Email. Cualquiera sirve como PK mínima; elegimos **Legajo** (las otras tres quedan como claves alternativas, con restricción de unicidad).

c) Pasaje a 3FN

La única separación real es la oficina: `Piso_Oficina` depende de `Id_Oficina`, no del empleado → tabla **Oficina**. Todo lo demás queda donde está.

Empleado

Legajo (PK)	DNI (único)	CUIL (único)	Nombre	Email (único)	Id_Oficina (FK)

Oficina

Id_Oficina (PK)	Piso

d) ¿Los atributos únicos «repetidos» violan alguna forma normal?

No. DNI, CUIL y Email dependen de la clave completa y no transitivamente: son identificadores alternativos del mismo objeto, no redundancia. El error típico acá es «descomponer de más» y sacar DNI/CUIL/Email a otras tablas: eso no arregla nada y complica el modelo.

Ejercicio 9 — Oferta de Cursos

a) Dependencias funcionales (una vez atomizados los datos)

- Cod_Curso → Nombre_Curso, Aula
- Aula → Capacidad_Aula
- Id_Docente → Nombre_Docente

b) Clave primaria mínima

De la tabla dada, **Cod_Curso** (hay una fila por curso).

c) Pasaje a 3FN

El problema principal acá son las **listas dentro de una celda** (docentes, teléfonos, horarios): ni siquiera se cumple 1FN. Cada lista se convierte en una tabla:

- Varios docentes por curso y varios cursos por docente → N:M → tablas **Docente** y **Curso_Docente**.
- Varios teléfonos por docente → tabla **Telefono_Docente**.
- Varios horarios por curso → tabla **Horario_Curso**.
- Y la capacidad depende del aula, no del curso → tabla **Aula**.

Aula

Id_Aula (PK)	Capacidad

Curso

Cod_Curso (PK)	Nombre	Id_Aula (FK)

Docente

Id_Docente (PK)	Nombre

Telefono_Docente

Id_Docente (PK, FK)	Telefono (PK)

Curso_Docente

Cod_Curso (PK, FK)	Id_Docente (PK, FK)

Horario_Curso

Cod_Curso (PK, FK)	Dia (PK)	Hora_Inicio (PK)	Hora_Fin

> Noten que `Telefonos_Docentes` era **doblemente** multivaluado: varios docentes y, por cada uno, varios teléfonos — el formato con `/` y `;` del ejemplo era la pista. Por eso el teléfono cuelga de `Docente` y no de `Curso`.

Ejercicio 10 — Logística

a) Dependencias funcionales

- Cod_Paquete → Nro_Envio, Peso_Kg
- Nro_Envio → Fecha, DNI_Cliente, CP_Destino, Patente_Camion, DNI_Chofer
- DNI_Cliente → Nombre_Cliente
- CP_Destino → Localidad_Destino
- Localidad_Destino → Provincia_Destino
- Patente_Camion → Modelo_Camion
- DNI_Chofer → Nombre_Chofer, Categoria_Licencia

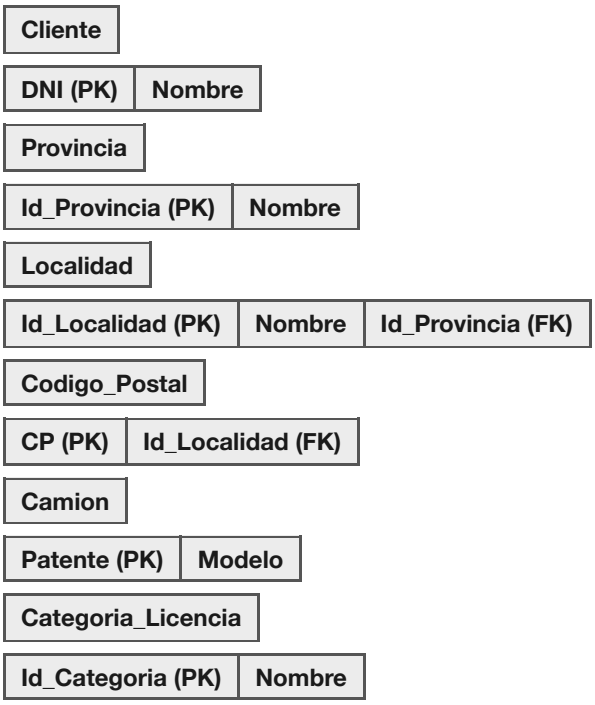
b) Clave primaria mínima

Cod_Paquete, solo. La tentación es (Nro_Envio, Cod_Paquete), pero como el código de paquete es único en todo el sistema y cada paquete pertenece a un único envío, `Cod_Paquete → Nro_Envio`: el paquete solo ya determina toda la fila. Una clave que contiene atributos innecesarios no es mínima.

c) Pasaje a 3FN

Todo lo que depende del envío (y no del paquete) se va con el envío, y de ahí cada cosa con su dueño:

- Fecha, destino, camión, chofer y cliente dependen del envío → tabla **Envio**.
- El nombre depende del DNI del cliente → tabla **Cliente**.
- El CP determina la localidad y la localidad la provincia → tablas **Codigo_Postal**, **Localidad** y **Provincia** (cadena de dos pasos, igual que médico–especialidad en el ejercicio 6).
- El modelo depende de la patente → tabla **Camion**; nombre y licencia dependen del chofer → tablas **Chofer** y **Categoria_Licencia**.
- Del paquete quedan su código, su peso y a qué envío pertenece.



Chofer

DNI (PK)	Nombre	Id_Categoria (FK)

Envio

Nro_Envio (PK)	Fecha	DNI_Cliente (FK)	CP_Destino (FK)	Patente_Camion (FK)	DNI_Chofer (FK)

Paquete

Cod_Paquete (PK)	Nro_Envio (FK)	Peso_Kg

> Un detalle más: el chofer del envío es una FK de `Envio`, no un dato del camión — el enunciado avisa que puede no ser el habitual. Si además quisiéramos registrar el chofer habitual, iría como FK en `Camion`: son dos hechos distintos.

Parte 3 — SQL

Ejercicio 11 — Completar la query

a)

```
SELECT c.nombre,
       COUNT(DISTINCT p.id_pedido) AS cant_pedidos,
       SUM(i.cantidad * pr.precio) AS total_gastado
FROM cliente c
JOIN pedido p       ON p.id_cliente  = c.id_cliente      -- (1)
JOIN item_pedido i  ON i.id_pedido   = p.id_pedido       -- (2)
JOIN producto pr    ON pr.id_producto = i.id_producto    -- (3)
GROUP BY c.nombre                                        -- (4)
HAVING COUNT(DISTINCT p.id_pedido) >= 2                  -- (5)
ORDER BY total_gastado DESC;                             -- (6)
```

En (5) también es válido `HAVING cant_pedidos >= 2` (SQLite permite usar el alias). La condición va en `HAVING` y no en `WHERE` porque filtra sobre un agregado, que solo existe después de agrupar.

b) Después de los tres JOIN hay **una fila por ítem**, no por pedido: un pedido con 3 productos aparece 3 veces. `COUNT(p.id_pedido)` contaría ítems; con `DISTINCT` se cuenta cada pedido una sola vez. `total_gastado` en cambio no necesita corrección: justamente queremos sumar todos los ítems.

c) Sin el `DISTINCT` la columna daría **Bruno 4, Ana 5, Carla 4**: la cantidad de renglones de sus pedidos (salida verificada en SQLite).

Ejercicio 12 — Del resultado a la query

a)

```
SELECT pr.nombre,
       SUM(i.cantidad)             AS unidades,
       SUM(i.cantidad * pr.precio) AS facturacion
FROM producto pr
JOIN item_pedido i ON i.id_producto = pr.id_producto
GROUP BY pr.nombre
HAVING SUM(i.cantidad) >= 3
ORDER BY unidades DESC, pr.nombre;
```

Puntos que había que detectar:

- `unidades` es `SUM(cantidad)`, no `COUNT(*)`: el Café americano aparece en 3 pedidos pero son 4 unidades (2+1+1).
- El desempate del orden: Cappuccino, Jugo de naranja y Tostado empatan en 3 unidades y salen alfabéticamente → `ORDER BY unidades DESC, pr.nombre`. Sin el segundo criterio, el orden entre empatados no está garantizado.

b) Porque es un filtro sobre un **agregado**: `SUM(i.cantidad)` recién existe después de agrupar. `WHERE` filtra filas individuales antes del `GROUP BY`; `HAVING` filtra grupos después.

c) No. Un `LEFT JOIN` serviría para conservar productos sin ventas (como el Té verde, que quedaría con 0), pero esos productos igual quedan eliminados por el `HAVING SUM(i.cantidad) >= 3`. Con el `JOIN` común el resultado es el mismo.

Ejercicio 13 — Predecir el resultado

a) Salida real (5 filas — Damián **aparece**, con 0):

nombre	cantidad
Ana	3
Bruno	2
Carla	2
Elena	1
Damián	0

Por el `LEFT JOIN`, Damián (sin pedidos) genera una fila con todas las columnas de `pedido` en NULL. `COUNT(p.id_pedido)` **no cuenta los NULL**, por eso da 0. Y por el orden (`cantidad DESC, nombre`) queda último.

b) Con `COUNT(*)` la única fila que cambia es la de Damián: pasa de 0 a **1**, porque `COUNT(*)` cuenta filas del resultado del join, incluida la fila «huérfana» de NULLs. Además cambia el orden: Damián deja de estar último y empata con Elena (salida real: Ana 3, Bruno 2, Carla 2, Damián 1, Elena 1). Es la diferencia clave entre `COUNT(*)` y `COUNT(columna)` combinada con `LEFT JOIN`.

Ejercicio 14 — Encontrar el error

a) La columna mala es `cant_pedidos`: los valores correctos son **Lucas 2, Diego 3, Sofía 2, Marta 1** (se verifica contando en la tabla `pedido`: Diego atendió 101, 104 y 106, etc.). `total_facturado` está bien: la sospecha del analista era correcta en ambos casos.

b) Al hacer JOIN con `item_pedido`, cada pedido se repite una vez por ítem (fan-out). `COUNT(p.id_pedido)` cuenta esas repeticiones: los pedidos de Sofía (102 y 105) tienen 2 ítems cada

uno → 4. La suma, en cambio, **debe** recorrer todos los ítems, por eso `total_facturado` no se ve afectado.

c) Basta con agregar `DISTINCT`:

```
SELECT e.nombre,
       COUNT(DISTINCT p.id_pedido) AS cant_pedidos,
       SUM(i.cantidad * pr.precio) AS total_facturado
FROM empleado e
JOIN pedido p      ON p.id_empleado = e.id_empleado
JOIN item_pedido i ON i.id_pedido   = p.id_pedido
JOIN producto pr   ON pr.id_producto = i.id_producto
GROUP BY e.nombre
ORDER BY total_facturado DESC;
```

nombre	cant_pedidos	total_facturado
Lucas	2	25500
Diego	3	18600
Sofía	2	13200
Marta	1	8200

d) Camila no atendió ningún pedido y el `JOIN` (inner) descarta a los empleados sin coincidencias: no tiene ninguna fila en `pedido` con su `id_empleado`.

e) Para incluirla hay que encadenar `LEFT JOIN` — y tienen que ser **todos** LEFT: si el segundo o el tercero son inner, las filas de Camila (con NULLs) se vuelven a perder.

```
SELECT e.nombre,
       COUNT(DISTINCT p.id_pedido) AS cant_pedidos,
       SUM(i.cantidad * pr.precio) AS total_facturado
FROM empleado e
LEFT JOIN pedido p      ON p.id_empleado = e.id_empleado
LEFT JOIN item_pedido i ON i.id_pedido   = p.id_pedido
LEFT JOIN producto pr   ON pr.id_producto = i.id_producto
GROUP BY e.nombre
ORDER BY total_facturado DESC;
```

nombre	cant_pedidos	total_facturado
Lucas	2	25500
Diego	3	18600
Sofía	2	13200
Marta	1	8200
Camila	0	NULL

Detalle fino: el total de Camila sale `NULL` (no 0), porque `SUM` sobre un grupo sin valores no NULL devuelve NULL.

Ejercicio 15 — Escribir desde cero

a) Self-join con `LEFT` para no perder a Marta:

```
SELECT e.nombre AS empleado,
       j.nombre AS jefe
FROM empleado e
LEFT JOIN empleado j ON j.id_empleado = e.id_jefe
ORDER BY e.nombre;
```

empleado	jefe
Camila	Diego
Diego	Marta
Lucas	Diego
Marta	NULL
Sofía	Marta

La fila de Marta devuelve `NULL` en la columna `jefe`. Con un `JOIN` común, Marta directamente desaparecería del resultado. Las dos «copias» de la tabla necesitan alias distintos (`e` y `j`).

b) `LEFT JOIN` + `IS NULL`:

```
SELECT pr.nombre, pr.categoria
FROM producto pr
LEFT JOIN item_pedido i ON i.id_producto = pr.id_producto
WHERE i.id_pedido IS NULL;
```

nombre	categoria
Té verde	Cafetería

c) Bonus:

```
SELECT e.nombre || ' reporta a ' || j.nombre AS frase
FROM empleado e
LEFT JOIN empleado j ON j.id_empleado = e.id_jefe
ORDER BY e.nombre;
```

frase
Camila reporta a Diego
Diego reporta a Marta
Lucas reporta a Diego
NULL
Sofía reporta a Marta

La fila de Marta da **NULL entera**: concatenar cualquier cosa con NULL da NULL (`'Marta' || ' reporta a ' || NULL → NULL`). NULL no es una cadena vacía: se «propaga» por casi todas las operaciones.